CS24301 — THEORY (3.0 CR) MODULE III NOTES BIT MESRA NEP SCHEME

Module 3: Semantic Analysis & Type Systems

SDD, SDTS, Synthesized/Inherited, Type Checking & Symbol Tables

Module III: Semantic Analysis & Type Checking — Complete Syllabus Topics

1. Role of Semantic Analysis & Static vs. Dynamic Checking
2. Syntax-Directed Definitions (SDD) & Semantic Rule Formulations
3. Synthesized Attributes vs. Inherited Attributes Formal Properties
4. S-Attributed Definitions & Bottom-Up LR Parsing Implementations
5. L-Attributed Definitions & Depth-First Left-to-Right Traversal Orders
6. Dependency Graphs & Topological Sorting for Attribute Evaluation
7. Syntax-Directed Translation Schemes (SDTS) & Action Placement Rules
8. Type Systems, Type Expressions & Static Type Checker Architecture
9. Type Equivalence (Structural vs. Name Equivalence Algorithms)
10. Type Conversions (Implicit Coercions vs. Explicit Casting Mechanics)
11. Symbol Table Organizations (Block Scoping with Chained Hash Tables)
12. Comprehensive Solved BIT Mesra & GATE Question Bank (8 Solved Problems)

Topic 1 & 2: Syntax-Directed Definitions (SDD) & Attribute Types

A **Syntax-Directed Definition (SDD)** is a context-free grammar augmented with attributes and semantic rules. Attributes are associated with grammar symbols, and semantic rules are associated with production rules.

Attribute Class	Formal Mathematical Formulation & Value Flow	Parsing & Evaluation Invariants
1. Synthesized Attribute	Computed strictly from the attribute values of the node's children in the parse tree (or lexical values): $A.s = f(Y_1.a_1, Y_2.a_2, \ldots, Y_k.a_k) \quad \text{for production } A \to Y_1 Y_2 \ldots Y_k$	Flows strictly **Bottom-Up**; naturally evaluated during shift-reduce / LR parsing using an attribute stack.
2. Inherited Attribute	Computed from the attribute values of the node's parent and/or left siblings: $Y_j.i = f(A.a, Y_1.a_1, \ldots, Y_{j-1}.a_{j-1}) \quad \text{for symbol } Y_j \text{ in } A \to Y_1 \ldots Y_k$	Flows **Top-Down** and **Left-to-Right**; passes context (e.g., base data types) down into declarations.

Topic 3 & 4: S-Attributed vs. L-Attributed SDD

Structural Classifications

- **S-Attributed SDD:** An SDD that uses **only synthesized attributes**. Can be evaluated during bottom-up parsing (e.g., LR parser) by maintaining an attribute value stack in parallel with the parser state stack.
- **L-Attributed SDD:** An SDD where every attribute is either synthesized, or inherited with the restriction that for production $A \to X_1 X_2 \dots X_n$, an inherited attribute of X_j depends only on:

1. Attributes of the parent A.
2. Attributes of symbols to the left of X_j ($X_1, X_2, \dots, X_{j-1}$).

Every S-Attributed SDD is strictly an L-Attributed SDD ($S\text{-Attributed} \subset L\text{-Attributed}$).

🏛️ Step-by-Step Solved Problem: Desktop Calculator S-Attributed SDD

Grammar Production	Semantic Evaluation Rule
$L \to E\ \mathbf{n}$	$\text{print}(E.\text{val})$
$E \to E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$
$E \to T$	$E.\text{val} = T.\text{val}$
$T \to T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$
$T \to F$	$T.\text{val} = F.\text{val}$
$F \to (E)$	$F.\text{val} = E.\text{val}$
$F \to \mathbf{digit}$	$F.\text{val} = \mathbf{digit}.\text{lexval}$

Evaluates expression `(3 + 4) * 5\n` bottom-up in a single LR parsing pass to yield `35`.

Topic 6: Dependency Graphs & Topological Evaluation Orders

A **Dependency Graph** represents the data-flow constraints between attribute instances in a parse tree:

For each node attribute $X.a$, there is a corresponding graph vertex.
A directed edge $X.a \to Y.b$ indicates that the semantic rule for $Y.b$ requires the value of $X.a$.
If the dependency graph contains **no directed cycles**, a valid evaluation order is given by any **Topological Sort** of the graph.

Topic 8 & 9: Type Systems & Type Equivalence

1. Structural vs. Name Equivalence:

Name Equivalence: Two types are equivalent if and only if they share the exact same named type identifier.

```
typedef struct { int x, y; } PointA;
typedef struct { int x, y; } PointB;
// Under Name Equivalence: PointA ≠ PointB (Strict)
```

Structural Equivalence: Two types are equivalent if they have identical internal constituent structures.

```
// Under Structural Equivalence: PointA == PointB (Both contain two integer fields)
```

2. Type Conversions & Coercion Rules:

Widening Conversions (Implicit): Preserve numeric precision (e.g., `char` → `int` → `float` → `double`). Automatically inserted by compiler.
Narrowing Conversions (Explicit): May lose precision or overflow (e.g., `double` → `int`). Require explicit type cast syntax.

Topic 11: Symbol Table Organizations (Block Scoping with Hash Tables)

Figure 3.1: Hierarchical Chained Hash Tables for Nested Lexical Block Scoping

🏛️ Top BIT Mesra Exam Questions & Answers (Module III)

Q1. Differentiate between S-Attributed and L-Attributed SDD with examples and evaluation strategies. (8 Marks)

1. **S-Attributed SDD:** Uses only synthesized attributes. Evaluated strictly bottom-up during LR parsing using an attribute value stack without needing an explicit syntax tree.
2. **L-Attributed SDD:** Allows synthesized and inherited attributes with the restriction that inherited attributes of X_j in $A \to X_1 X_2 \dots X_n$ depend only on attributes of parent A and left siblings $X_1, \dots, X_{j-1}$. Evaluated in a single depth-first, left-to-right tree traversal.

Q2. Write an SDD to translate type declarations like `int a, b, c;` into symbol table entries. (8 Marks)

Production	Semantic Rule
$D \to T\ L$	$L.\text{in} = T.\text{type}$
$T \to \textbf{int}$	$T.\text{type} = \text{integer}$
$T \to \textbf{float}$	$T.\text{type} = \text{real}$
$L \to L_1,\ \textbf{id}$	$L_1.\text{in} = L.\text{in};\ \text{addType}(\textbf{id}.\text{entry}, L.\text{in})$
$L \to \textbf{id}$	$\text{addType}(\textbf{id}.\text{entry}, L.\text{in})$

Uses inherited attribute $L.\text{in}$ to propagate base type downward to all identifier instances.